
Directing Intelligence with Governed Agents

Jackson Stephens

claude_preflight • jackson@stephens.sh • August 2026 • Technical Report • v0.9.9

Abstract

Modern frontier agents are great at building features and poor at building components within the greater context of a project. Given a scoped task they produce decent code for a feature. Asked to hold the rest of the project in view while they do it, so that the feature integrates cleanly with everything already built, they struggle. What that costs is contradicted decisions, the same mistake made in a cycle, tokens spent rebuilding context the agent already had, and a great deal of avoidable headache. It also costs safety: an agent that cannot see the whole will still run a destructive command if nothing stops it. Those are not model-quality problems and a better model does not remove them. This report describes a project that aids my own agentic development and, along the way, shows the considerations I am making while working in that space. It attacks four of those problems mechanically: authorizing every tool call before it runs, requiring a human to ratify changes to the rules themselves, re-injecting the design spine after context compaction, and making the system's claims about itself executable so they cannot quietly drift. The enforcement layer is a fail-closed policy gate built on Cedar, with 260 pinned verdicts and an adversarial suite that prints the bypasses it has not closed. Two results here are negative and are reported unchanged. A gated versus un-gated trial against indirect prompt injection returned no difference, because the model refused every attack unaided; and the anti-forgetting bundle scored below simply letting the agent read the repository. Both contradict the stronger claim this project began with, so that claim has been narrowed: alignment is probabilistic and a policy gate is deterministic, which matters most for weaker or untrusted models, for actions a model has no reason to refuse, and wherever an audit trail is required. Nothing here is complete or airtight. A static gate cannot see past a variable or across two tool calls, gate integrity currently rests on the agent's cooperation rather than the operating system, and sandboxing remains the layer beneath this one. The contribution is a set of named problems, partial mechanical answers to each, and a measurement discipline that keeps reporting the score when it goes down.

1. Introduction

A modern frontier agent will build you an excellent feature, with a good chance that the feature majorly alters, or fails to integrate elegantly with, the greater picture of your *project*. Handed a well-scoped unit of work, the model writes it competently, tests it, and explains it. Handed a codebase over weeks, the same model loses the shape of the thing: it duplicates a component that already exists, violates an invariant it helped write, reopens a decision that was settled in a document it can no longer see, and produces a system that is locally sensible everywhere and globally incoherent.

That gap is the subject of this report. It is not a capability gap and it does not close as models improve, because it is a property of how the work is delivered rather than how well any single step is performed. Two failure modes sit on either side of the model's competence, and neither is fixed by a better model.

The first is **forgetting**. An agent's working memory is a context window, and a long session overflows it. When the harness compacts that context, the agent keeps the recent diff and loses the *bigger picture*, the architecture it is supposed to preserve, the invariants it agreed to, the decisions that explain why the code looks the way it does. The result is an agent that re-litigates settled design questions, contradicts its own prior choices, and drifts away from the system it is meant to be building. This is not a knowledge gap the model can close; it is a state-management gap in the scaffold around it.

The second is **the absent guardrail**. An agent with shell access will, if nothing stops it, run a destructive command, read a secret, or rewrite the very rules meant to constrain it, not from malice but because the token that does

the harm looks like the token that does the work. The platforms that distribute these agents generally omit the governance layer: policy-as-code with human-gated approvals, class-aware provenance, and an audit trail of what was allowed and why. That omission is a product decision rather than an oversight, and it leaves each serious user to build the missing half.

claude_preflight addresses both with one install. It is not a research prototype built to make a point: it is a project that aids my own agentic development, and this report is an account of the considerations that project surfaces while I work in that space. This report began from a stronger organizing bet, **scaffold > model**, and the measurements in Section 7.2 did not support it: a frontier model refused every injected attack without any gate, and the anti-forgetting bundle scored below simply letting the agent read the repository. The surviving claim is narrower and is the one to take away: **alignment is probabilistic and a policy gate is deterministic**. The scaffold does not make a capable model more capable. It makes a bad outcome structurally impossible rather than merely unlikely, and it leaves a record of which. The rest of this report is what that means mechanically, and what it is worth when measured rather than asserted. I report the architecture of both halves, the assurance layer that pins them, and one case study (Section 7) whose only job is to show the guardrail is load-bearing rather than decorative.

2. The governed half: nothing harmful runs

Every tool call the agent makes, a shell command, a file write, an edit, is authorized *before it executes*. The enforcement path is short and fail-closed.

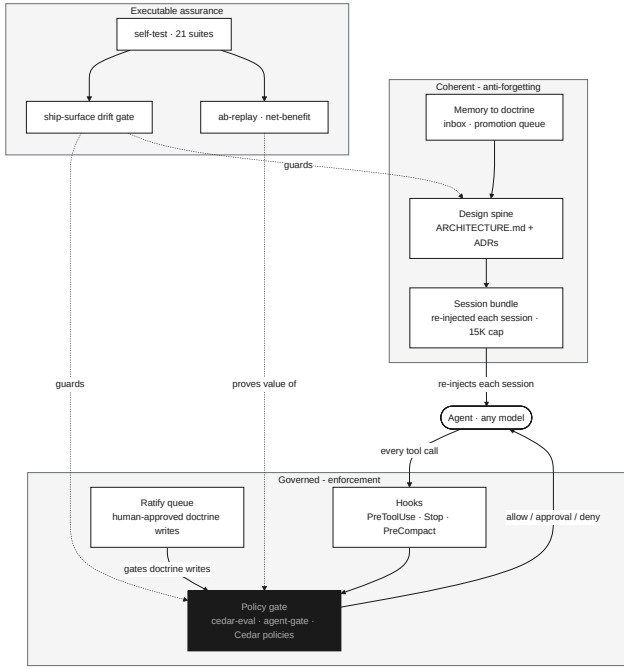


Figure 1: The system at a glance. The *governed* half gates every action through the Cedar policy engine; the *coherent* half re-injects the design spine each session; the *executable-assurance* layer guarantees neither can silently drift. The emphasized node is the policy gate, the single load-bearing enforcement point every tool call passes through.

2.1. The policy gate

The gate is a three-stage pipeline. A PreToolUse hook intercepts the call; `cedar-eval.sh` tokenizes it into context flags (is this a destructive command? a secret read? a write to a doctrine file?); and those flags are handed to **Cedar**, the sole policy engine, which returns a verdict against the policies in `governance/policy/cedar/`. Cedar was chosen as the one engine and Rego/OPA purged (ADR 0006) so there is a single place where a verdict is decided and a single regression guard over it.

Verdict	When	Effect
allow	benign shell, reads, a doctrine-free write	runs immediately
require approval	a write touching doctrine, or a gateable class	pauses for a human marker or a recorded council
deny	destructive, remote-pipe-to-shell, secret read	hard-blocked, never runs

Table 1: The three verdicts of the policy gate. The gate is load-bearing, not advisory: a *deny* class never runs.

The design invariant is that the gate is *load-bearing*, not advisory. When the Cedar binary is present the gate runs full evaluation; when it is absent it degrades to a tokenizer-only verdict that still blocks the dangerous classes and still allows

the bootstrap that installs the engine: degraded, never bricked. A gate that bricks the session on a missing dependency would be uninstalled within the hour; a gate that silently allows everything when the engine is missing would be theatre. The degraded path threads that needle. It blocks danger with the tokenizer alone and lights up a red self-test (`cedar_live`) until the real engine is back.

2.2. Hooks: the interception layer

Eleven hooks wire the gate and the anti-forgetting loop into the agent's lifecycle: PreToolUse (authorize), Stop (the Continuation Contract), PreCompact (synthesize before context is lost), SessionStart (inject the bundle). Every hook is fail-open by construction (trap 'exit 0' ERR, a `stop_hook_active` loop-guard, a per-hook disable switch) for one reason: a *coherence* mechanism that can wedge the session is worse than the forgetting it prevents. Enforcement fails closed; the machinery around enforcement fails open. That asymmetry is deliberate and it is the difference between a harness people keep installed and one they rip out.

2.3. The ratify queue

The gate can deny, and it can allow, but its third verdict, *require approval*, is where governance earns its keep. Some changes an agent may **propose** but must never **ratify** on its own authority: an edit to the constitution, to `AGENTS.md`, to the Cedar policy, to the gate engine itself, to `settings.json`, to a lockfile. These are the **strict class**. The agent stages the change into `.agent/ratify-queue/` and reminds the human to run `/preflight --ratify`; the human reviews the full diff and applies it. The agent never writes strict-class doctrine by any route, not through a sub-agent, not through a heredoc, not through a shell redirect that dodges the Write path. Section 5 makes the case for why this specific boundary, and not a looser one, is load-bearing.

3. The coherent half: nothing forgotten

Enforcement stops the agent from doing harm. Coherence stops it from doing damage of a subtler kind: quietly forgetting what it is building and drifting off the design.

3.1. The session bundle

At the start of every session, and after every compaction, a bundle is compiled and injected into the agent's context. It is capped at **15K tokens** and it carries five things: the mission, the architecture spine, the binding doctrine, the active state, and the hard constraints. It is not a document the agent might choose to read; it is re-confronted with the design spine before it takes its first action of the session. The cap is a feature. A bundle that grew without bound would itself get compacted away, so the discipline is to keep the re-injected spine small enough that it always survives.

The rule that governs this loop is that *every compaction must yield a sharper post-compact agent, not a more amnesiac one*. Compaction is treated as an opportunity to re-inject the spine, not a tax that erodes it. The bundle's fidelity to the actual spine is itself regression-pinned: an `amnesia_`

regression check fails if the injected Section 1b drifts from the first sixty lines of the real ARCHITECTURE.md, so the anti-forgetting mechanism cannot itself quietly forget.

3.2. Memory to doctrine

A durable instruction should be *enforced*, not repeated. When the human states a lasting preference, a rule, or a recurring correction, it is captured to memory/inbox.md and a promotion candidate is added to governance/PROMOTION_QUEUE.md **in the same response**. From there it can be promoted into a canonical doctrine document and, where appropriate, into runtime enforcement.

Promote, don't repeat. A rule the human has to ask for twice is a promotion failure; the harness treats it as a defect, not a normal cost of doing business. This is the mechanism by which a preference stated once becomes a constraint enforced always, closing the loop between “I told the agent” and “the agent cannot do otherwise.”

3.3. The coherence layer

Forgetting is not only about rules; it is about *design*. A landed feature can be locally correct and globally incoherent: it works, but it contradicts an invariant three subsystems away. The coherence layer (proposal 26) maintains the design spine (docs/ARCHITECTURE.md plus the ADRs), re-injects it at session start, and runs a coherence check when a feature lands so that architectural drift surfaces as a *finding* rather than a surprise discovered months later. An architectural-class change, including populating the spine in any new domain, requires an ADR. The maintained spine is the single source of truth for how the parts fit and what must stay true; the layer's job is to keep the agent reconciling against it instead of rediscovering it.

4. Executable assurance

Documentation drifts from reality; tests do not. The assurance layer is a set of checks that *fail on sight* when the governed or coherent half degrades, so a claim in this report is either backed by a check or it is not claimed (Table 2).

The **enforcement-proof trio** deserves emphasis because it answers the question a skeptic should ask: is the gate actually the thing doing the work, or is it decoration that happens to sit next to safe behavior? The mutation check neuters a detector and asserts a real verdict flips, proving the detector is causal. The causation check pins the deny to the specific policy that produced it. The adversarial check holds the gate under obfuscated inputs and, critically, discloses the bypasses that remain open rather than hiding them. That last discipline, a red-team suite whose green never means “airtight,” only “every known bypass is either closed or loudly documented,” is the exact posture the case study in Section 7 was forced to formalize.

The bundle dogfoods its own discipline.

claude_preflight is built under the governance it ships: the gate blocks edits to the gate (forcing the ratify flow), the drift test catches drift in the bundle's own ship surface, and the Continuation Contract forces an honest status at every stop. A governance layer that cannot

govern its own development is theatre. This one demonstrably is not, and the case study that follows is a recorded instance of the harness governing a change to itself.

5. The human-in-the-loop ratification model

The single most load-bearing rule in the system is short: **an agent may propose a strict-class change but may never ratify one**. It is worth being precise about why this specific boundary, drawn in this specific place, is what makes the whole harness trustworthy rather than merely tidy.

An agent that can edit its own constitution has no constitution. If the rules that constrain behavior are writable by the thing they constrain, the first inconvenient rule gets amended away, not through defiance, but through the ordinary path of an agent optimizing for the task in front of it and treating its own guardrail as one more obstacle to route around. The failure is not hypothetical: an agent asked to “make the tests pass” will, given the authority, weaken the test. So the strict class draws a bright line around exactly the artifacts that define authority itself, the constitution, AGENTS.md, the Cedar policy, the gate engine, settings.json, lockfiles, and puts a human on the far side of it.

The mechanism has three tiers, chosen by the *stakes* of the change, not by the agent's convenience. A change to the gate engine is validated **on a copy** first: the full self-test and golden regressions run against the proposed version, so a change that would break enforcement is caught before it touches the live tree. It is then reviewed by a **different vendor** than the one that authored it (a Codex/GPT reviewer for a Claude-authored change, and vice versa), because a single vendor reviewing its own work shares its own blind spots. Only then does it go to the **council**.

A council here is a recorded multi-lens review, not a metaphor: the change is argued from a fixed set of perspectives, security, doctrine, and strategy, each of which must return a verdict that is written to disk with its reasoning, and only a consensus stages the change. Its composition is chosen by the project's declared tier rather than by the agent's mood: a tier-1 project gets a second-vendor reviewer, tier 2 adds a devil's advocate whose job is to argue the change down, and tier 3 adds a security lens and a named human. The agent selects the tier from the stakes of the change and must justify the choice in the record. The critical limit is that a council can *stage*; it cannot *ratify*. Ratification of a strict-class change is human-only, with no exception for autonomous mode, sub-agent delegation, council unanimity, or claimed urgency.

This is not bureaucracy for its own sake. It is ratification *without stalling*: the agent stages and keeps working; the human applies on their own schedule. The inner loop stays fast because the agent never blocks waiting for approval; it queues the change and moves on. The human stays in control of doctrine because nothing in that queue enters the live tree until they review the full diff and apply it. The recorded history bears this out: on June 2026 a targeted /bin/rm fix passed the copy-validation and cross-vendor review and was ratified, while a broader regex normalizer proposed the same

Check	What it proves	Fails when
<code>self-test</code> (24 suites)	the whole bundle is internally consistent	any suite regresses
<code>golden_shell</code> , <code>golden_filesystem</code>	the gate returns the pinned verdict for every catalogued command/path	a verdict silently changes
<code>cedar_live</code>	the <i>real</i> Cedar binary is enforcing, not a stub	the gate is inert
<code>ship_surface_drift</code>	the drop-in payload contains no legacy-engine references	drift enters the shipped surface
<code>ab-replay</code>	the bundle stops more harm than a vanilla agent	the net benefit disappears
enforcement-proof trio	the gate is load-bearing: mutation flips a verdict, causation pins the deny to the policy, adversarial holds under obfuscation and <i>discloses</i> open bypasses	the gate stops mattering
<code>portability_check</code>	the runtime adapters cannot drift from what they wrap	a shim diverges or forges an approval

Table 2: The executable-assurance layer. Each check fails on sight when the governed or coherent half degrades. A claim in this report is either backed by one of these or it is not made.

week was **held** by the council for introducing a false positive. The mechanism caught a bad change to enforcement before it shipped. That is the whole point of drawing the line where it is drawn.

6. Adoption: giving a project its own rules

Everything above assumes the project already has rules worth enforcing. It usually does not, and generic rules are worse than none: an agent that is told to “follow best practices” has been told nothing, and a policy that gates nothing specific gets read once and routed around. So the harness has to produce a project’s doctrine before it can enforce it, and it has to do that on a codebase that already exists.

6.1. The intake interview

First installation runs a **twenty-question intake**: a bank of questions at `governance/onboarding/questions.md`. It is not a survey. It is the compiler input for the project’s policy, and every question declares where its answer lands.

One sentence on what the project does becomes a core invariant in the constitution. The declared *tier*—sandbox, shipping, high-stakes, or long-running autonomous—becomes `.agent/project-tier.yaml`. That file decides which governance artifacts are required and how large a council a change must clear. The blast radius of a bad unmonitored change lands in a risk register. The answer about what sensitive material the repository touches lands in a threat model and adds entries to the Cedar secrets allowlist, so it changes what the gate blocks at runtime.

The interview runs in two modes. Interactive mode walks the bank with the human. Autonomous mode fills every question it can from evidence already in the repository—manifests for the stack, git-log frequency and CI for project lifetime, a scan for `.env`, `secrets/`, and `credentials*` for sensitivity, and cross-project memory for the rest—then asks only what it could not answer. The human is spent on judgment calls, not transcription. Either way the generated constitution and `AGENTS.md` are strict-class artifacts: they are staged, and the human ratifies them before they are committed. The system does not get to write its own rules even on day one.

6.2. Walking into a project that already exists

A new project is the easy case and the rare one. The question that actually decides whether this is usable is whether you would be comfortable inviting a governed agent into a repository you have already been working in for months, with its own hooks, its own CI, and its own agent instructions that you do not want touched.

Adopting into an existing project is therefore a separate tool, `scripts/preflight-adopt.sh`, with a different contract: safety over completeness. It runs in three steps. It **inventories** the target read-only, reporting which governance-shaped assets already exist. It then **plans**, classifying every component of the bundle as ADD, CONFLICT, or SKIP and printing the classification without writing anything at all, so the first run is always a proposal. Only on an explicit `-apply` does it write, and then only the additive files; every conflict is backed up and *left in place* rather than silently overwritten. It refuses to adopt the bundle into itself, and it never wires the gate into your settings on your behalf: attaching enforcement stays a decision you make, with the hook edit in front of you.

That contract is tested rather than asserted. The drop-in gauntlet at `tests/dropin_gauntlet.sh` copies four unrelated real projects to temporary directories, adopts each one, and then fires destructive commands through the *real* hook with the adopted project as the working directory. All four pass: existing agent rules come out byte-identical, and the harmful commands are blocked in the adopted tree rather than only in the tree the gate was developed in. Confidence in inviting an agent into an existing codebase should come from exactly that sequence—a read-only plan you can reject, an apply that cannot clobber your doctrine, and a live verdict measured in your own repository—rather than from a claim in a README.

7. Case study: the secret-read gate hardening

Everything above is architecture. This section is the evidence. It is a single, measured, reproducible change to the gate itself, driven to convergence by four rounds of adversarial dual-vendor review, and it is the paper’s empirical spine because it shows both what the governance is worth *and* exactly where

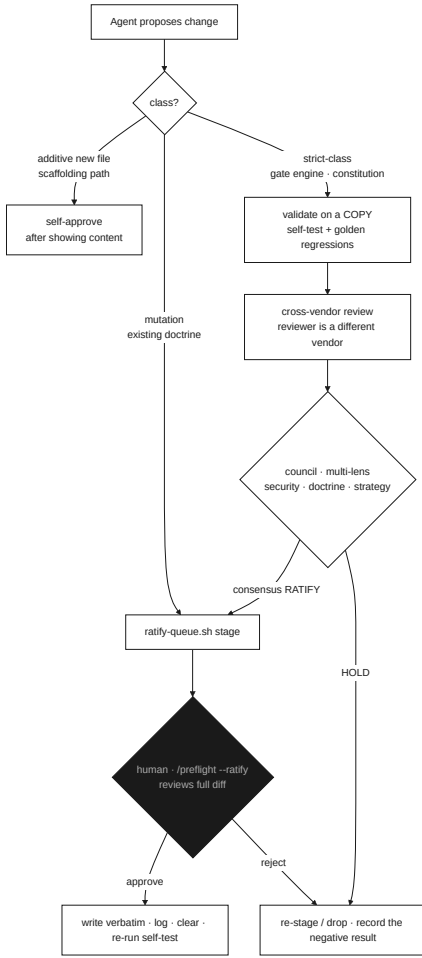


Figure 2: The class-aware ratification lifecycle. An additive new file in a scaffolding path is self-approved after its content is shown. A mutation of existing doctrine is staged for the human. A strict-class change to the gate engine or constitution escalates further: it is validated on a copy, cross-vendor reviewed, and councilled before it is even staged, so a flawed change to enforcement is caught before it can reach the human, let alone *main*. The emphasized node is the one authority that can ratify: the human.

its honest limit sits.

7.1. What was wrong

A pre-open-source audit found that the live shell policy gate did not deny reads of secret files, while advertising that it did. `cat ~/.ssh/id_rsa`, `base64 .env`, and `head ~/.aws/credentials` all returned *allow*. A staged fix existed, but it was itself bypassable and it over-blocked: it denied a read verb sitting harmlessly inside a commit message. The gap between the advertised protection and the live behavior is exactly the kind of silent drift the assurance layer exists to catch, and catching it *before* release is the whole reason a pre-ship audit runs the live gate rather than trusting the prose.

7.2. The measured result

The fix routes the secret-read detector through the *same* quote-aware normalizer the danger check already uses, matches the read verb in **command position** rather than anywhere in the

segment, and strips leading command wrappers so the real `argv0` surfaces. The before/after is measured, not asserted (Figure 3, Table 3).

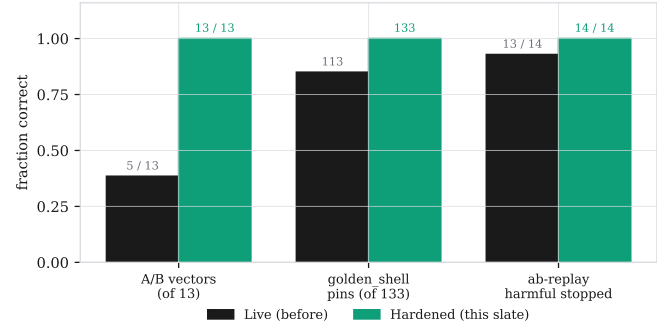


Figure 3: Measured A/B, live gate versus the hardened slate. Ink is the live gate before the fix; the emphasized (mint) bars are the hardened gate. A/B vector correctness moves from 5/13 to 13/13; the golden-shell regression set grows from 113 to 133 pinned cases and holds; ab-replay’s harmful-stopped count moves from 13/14 to 14/14. All values trace to `figures/ab_data.json`.

Metric	Live	Hardened
A/B vector correctness	5 / 13	13 / 13
golden_shell pins	113 / 113	133 / 133
golden_filesystem	81 / 81	81 / 81
ab-replay stopped	13 / 14	14 / 14
Benign false-positives	over-blocks	0 / 8
cedar_live enforce	PASS	PASS

Table 3: Headline before/after for the hardened slate. Every number traces to the reproduction commands in Section 11.

The design target was explicit and two-sided: **block when it counts, do not block diagnostics**. The hardened gate closes every secret-read and wrapper form *while* keeping a read verb sitting inside inert prose, a commit message, an echoed sentence, allowed. The old detector over-blocked those; a governance tool that cries wolf on `git commit -m '...cat ~/.ssh/id_rsa...'` trains its user to disable it, which is a security failure dressed as a security feature. Table 4 enumerates the classes.

7.3. How the detector decides

The hardened detector is a normalizer, not a regex list. It splits the command on separators, expands `${IFS}`, applies leading `VAR=` assignments, strips command wrappers (`env`, `command`, `nohup`, `timeout N`, `nice -n N`, with their per-wrapper operand flags), and recurses into `sh -c / bash -c / eval` up to a bounded depth. Only then does it ask whether a read verb sits in *command position* against a secret path, the distinction that lets it deny `bash -c 'cat KEY'` while allowing `echo cat .env` (Figure 4).

7.4. Why the review took four rounds

Hardening a static, non-executing gate against shell obfuscation is not one fix but a sequence of them. The dangerous call

Class	Command	Want	Live	Hardened
secret-read (direct)	<code>cat ~/.ssh/id_rsa</code>	deny	allow ×	deny ✓
secret-read (dotenv)	<code>base64 .env</code>	deny	allow ×	deny ✓
secret-read (wrapper)	<code>bash -c 'cat ~/.ssh/id_rsa'</code>	deny	allow ×	deny ✓
secret-read (env-wrap)	<code>env cat ~/.aws/credentials</code>	deny	allow ×	deny ✓
secret-read (flag-op)	<code>timeout 5 cat ~/.ssh/id_rsa</code>	deny	allow ×	deny ✓
secret-read (redirect)	<code>cat < ~/.ssh/id_rsa</code>	deny	allow ×	deny ✓
diagnostic (commit)	<code>git commit -m '...cat KEY...'</code>	allow	allow ✓	allow ✓
diagnostic (echo verb)	<code>echo cat .env</code>	allow	allow ✓	allow ✓
diagnostic (grep)	<code>grep -r id_rsa src/</code>	allow	allow ✓	allow ✓
danger (rm)	<code>rm -rf /</code>	deny	deny ✓	deny ✓
danger (wrapper)	<code>env rm -rf /</code>	deny	deny ✓	deny ✓
danger (IFS)	<code>rm\${IFS}-rf\${IFS}/</code>	deny	allow ×	deny ✓
danger (b64 to sh)	<code>echo ... base64 -di bash</code>	deny	allow ×	deny ✓

Table 4: Class-by-class verdicts, want versus live (before) versus hardened (after). ✓ marks a verdict matching intent, × a mismatch. The hardened gate closes every secret-read, wrapper, and danger-obfuscation form while leaving the three diagnostic cases (a read verb inside inert prose) correctly allowed. KEY abbreviates `~/.ssh/id_rsa` in the commit row.

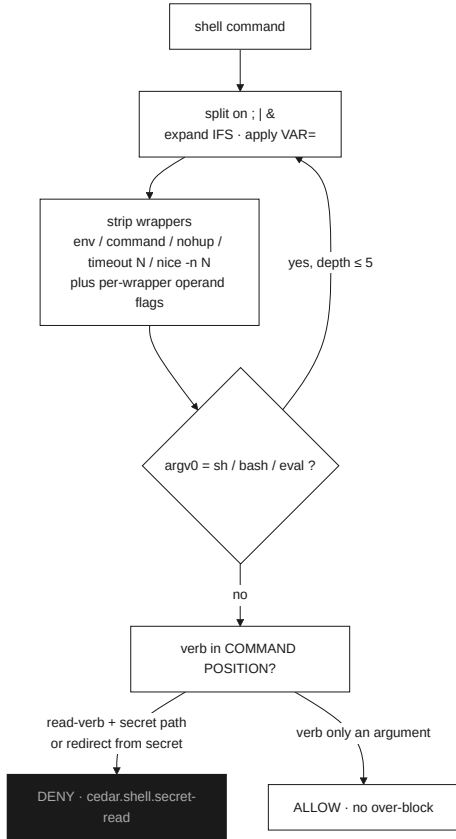


Figure 4: The detection flow. The command is normalized and unwrapped; a wrapper whose `argv0` is a shell recurses (bounded at depth 5); otherwise the verb is checked in command position. The emphasized node is the deny: a read verb, in command position, against a secret path, or a redirect from one.

is fixed; the number of syntactic routes to it is not, and each round of review surfaces another route rather than exhausting the set. The honest way to run that process is to keep score.

Each round of the dual-vendor review found a new evasion; each was closed and pinned with a regression case (Table 5).

7.5. The finding: convergence, not perfection

After round four the remaining open vectors are exactly one class: the **execution-requiring** ones, `$(echo cat) KEY`, `python -c "open('KEY')"`, copy-then-read, deeply-nested mixed-quote wrappers. A gate that does not execute the command *cannot* resolve these. They are identical in kind to the disclosed gaps the danger path (`rm -rf /`) already ships and already documents. This is the architectural finding the case study forced, and it generalizes past this one detector.

A static, non-executing gate cannot be airtight, and pretending otherwise is the real failure. The correct target is not a sandbox. It is: close every statically-recognizable form with zero false-positives, and disclose the execution-requiring residuals at parity with the danger path. Defense-in-depth that raises attacker cost, honestly bounded, not a wall claimed to be higher than it is.

Every residual is tested. `KNOWN_GAPS` in `tests/gate_adversarial_check.py` enumerates the open vectors so that a green suite is never misread as an airtight one. That is the discipline generalized: the value of the gate is not that it is perfect but that its imperfection is *known, bounded, and pinned*. The attacker pays a real cost, and the defender is never lied to about how high the wall actually is.

8. Significance

The case study is small on purpose, and its significance is out of proportion to its size for three reasons.

It shows the governance is load-bearing, measured. The move from 5/13 to 13/13 A/B correctness is not a claim about intent; it is a verdict count from the live gate, reproducible from committed artifacts. A governance layer that cannot

Round	Reviewer found	Resolution
1	<code>bash -c 'cat KEY'</code> undisclosed bypass; <code>echo cat .env</code> new false-positive	route through the normalizer; match the verb in command position
2	<code>env / command / nohup / timeout</code> wrapper prefixes; depth-3 nesting	add the wrapper-strip step; raise recursion depth 3 to 5
3	flag-operand wrappers (<code>env -u FOO cat, xargs -n 1 cat</code>); glued <code>cat<KEY</code>	per-wrapper operand table; space the bare <code><</code> redirect
4	<code>env -S eval-via-env</code> ; separated long-form flags; escaped- <code><</code> false-positive	fix the false-positive; disclose the execution-requiring tail

Table 5: The four review rounds. Each reviewer pass found a new evasion; each was closed and pinned. Round four ends not in a claim of airtightness but in a disclosed residual, the execution-requiring tail.

produce a number like this, a before, an after, a test that fails when the after regresses, is asking to be taken on faith, and the entire thesis of this project is that faith is the wrong currency.

It shows the ratification model working on the hardest case. The change under study is a change to the *gate engine itself*: the maximally strict class. It went through copy-validation, cross-vendor review, and a council, and it is staged for a human rather than self-applied. The harness governed a change to its own enforcement without letting the enforcer rewrite itself. That is the recorded proof that the human-in-the-loop boundary in Section 5 is real machinery, not a slogan.

It produces a reusable engineering posture. “Close every statically-recognizable form with zero false-positives; disclose the execution-requiring residuals at parity with the danger path” is a design rule any policy-gate author can adopt. It replaces the two common failure modes, claiming airtightness (dishonest) and over-blocking to feel safe (self-defeating), with a bounded, testable middle. The `KNOWN_GAPS` discipline is the portable artifact: a green test that is honest about what it does not cover.

9. Limitations and honest boundary

A governance tool that overclaims is worse than none, because it converts vigilance into false comfort. The honest boundary of this system has three parts.

- **The gate is not a sandbox.** By the case study’s own finding, a static non-executing gate cannot resolve the execution-requiring bypass class. The disclosed residuals in `KNOWN_GAPS` are real open vectors, tracked for closure, not hidden. The gate raises attacker cost and closes every statically-recognizable form; it is defense-in-depth, and it should be one layer among several (filesystem permissions, credential hygiene, least-privilege execution), never the only wall.
- **The governance number is measured; the productivity number is not.** The guardrail and anti-forgetting claims here are backed by tests: the gate stops the harmful actions a vanilla agent executes, drift is mechanically prevented, self-test is green. What is *not* yet measured is the compounding agent-task value, whether the harness makes an agent deliver features better, recover from compaction

better, waste fewer turns. That requires live A/B sessions (`scripts/ab-validate.sh`: defined, runnable, and honestly unrun as of this writing). So the claim is bounded precisely: the guardrails and anti-forgetting are real and measured; the productivity uplift is designed and plausible but not yet empirically established. Install it for the governance and coherence, not on a promise of proven net speed.

- **Success is not a benchmark.** This project has no leaderboard to top. Its success condition is “drops into any project and keeps the agent coherent and governed,” which is a property of adoption and durability, not a score. That makes it harder to advertise and easier to trust: there is no metric to game, only a harness that either holds up under its own discipline or does not. The strongest evidence on offer is that it is built under the governance it ships, and that when a hole was found in its own gate, the fix was measured, adversarially reviewed, and staged for a human rather than quietly self-applied.

10. Conclusion

The value in an agentic stack is the harness around the model, not the model, and governance and anti-forgetting are the two parts the platforms do not ship. `claude_preflight` makes an agent governed (nothing harmful runs) and coherent (nothing forgotten) with one install, and pins both with an executable assurance layer so neither drifts silently. The July 2026 secret-read hardening is the measured proof that the guardrail is load-bearing: A/B correctness moved from 5/13 to 13/13 at zero false-positives, pinned by a 113 to 133-case regression set, on a change to the gate engine itself that was staged for a human rather than self-applied. The finding it forced is the transferable one: a static gate cannot be airtight, so close every statically-recognizable form with zero false-positives and disclose the execution-requiring residuals at parity with the danger path. A green test that hides its gaps is the real failure; a green test that enumerates them is the tool.

11. Sandboxing is the layer beneath this one

A policy gate reads a command before it runs and decides. That is useful and it is also, by construction, a static analysis

of a string. It cannot resolve what a variable will hold, what a downloaded script will contain, or what a second tool call will do with a file the first one wrote. This report documents several such residuals rather than claiming they do not exist.

The mitigation is not a better parser. It is running the agent inside a boundary the agent cannot reason its way past. In practice that means three things, in descending order of value:

1. **A container or virtual machine.** Give the agent a disposable filesystem and no route to the host. Everything the gate might miss is then bounded by what the container can reach. This is the single highest-value control and it is independent of any harness.
2. **Credentials the agent never holds.** A secret that is not mounted cannot be exfiltrated by any command shape. Broker access rather than granting it; scope tokens to the task.
3. **A user the agent cannot escalate from.** The enforcement machinery should be owned by a different account than the one the agent runs as. Measured on my own machine, the agent's user is in the sudo group, which means it could undo any hardening applied to the gate. The harness reports this state rather than assuming the good case.

The honest ordering is therefore: sandbox first, policy gate second, model disposition third. This project supplies the second layer and a set of tests that keep it honest. It does not supply the first, and a reader who adopts only the gate has bought defense in depth, not a boundary.

Reproducibility

Every component named in this report is in the repository:

Path	Subsystem
governance/policy/cedar/	policy rules
scripts/cedar-eval.sh	tokenizers and evaluator
scripts/agent-gate.sh	class dispatcher
hooks/	lifecycle hooks
scripts/compile-session-start.sh	session bundle
governance/PROMOTION_QUEUE.md	memory to doctrine
docs/ARCHITECTURE.md	design spine
methodology/decisions/	the ADR record
docs/EVIDENCE.md	the claim ledger
tests/	24 self-test suites

Table 6: Where each subsystem lives. Every headline number in this report traces to a named test in `tests/` or to the claim ledger.

Every figure in this report is generated rather than drawn. Plots and diagrams both live under `paper/figures/`: the data figure via `make_figures.py`, the Mermaid diagrams via `render_mermaid.py`. Reproduce the measured results with:

```
./scripts/measure/self-test.sh # 24/24
python3 tests/golden_shell.py # 179/179
python3 tests/golden_filesystem.py # 81/81
python3 tests/gate_adversarial_check.py
# 36/36 enforced, 4 residuals disclosed
python3 tests/privilege_separation_check.py
# reports the gate's integrity tier
python3 tests/cedar_live.py # cedar PASS
scripts/measure/ab-replay.sh # 14/14 stopped, 0 FP
```

Two of those commands refuse to let a green run mean more than it does. `gate_adversarial_check` prints the bypasses that remain open rather than hiding them, and `privilege_separation_check` fails the build if this paper or the README claims stronger guarantees than the machine can demonstrate. A green run that hides its gaps is the failure mode; a green run that enumerates them is the tool.

Closing statement

This is an imperfect solution, and it will iterate. I am glad to have shared some of the major pieces I have seen work in practice: a fail-closed policy gate, human ratification of the rules themselves, a design spine re-injected after compaction, and claims that fail in public when they drift. None of that is finished, and none of it replaces the layer beneath - a sandbox and least-privilege credentials still do work this harness cannot. The useful distinction is not smart versus dumb models. It is probabilistic refusal versus a deterministic deny, with an audit trail either way. Alignment is probabilistic. A policy gate is not. This project does not make a capable model more capable; it makes some bad outcomes structurally impossible rather than merely unlikely, and it leaves a record of which. Where the measurements went the other way, the claim was narrowed rather than defended. That narrowing is the point of measuring. The goal, at the end of the day, is value at every stage of a project - beginning, middle, or late - so that working with agents feels governable: directing intelligence with current methods, under rules you can see, while the methods themselves keep changing.

References

1. Cedar Policy Language. Open-source authorization engine, originally developed at Amazon Web Services.
<https://www.cedarpolicy.com/>
<https://github.com/cedar-policy/cedar>
2. Cedar: A Language for Expressive, Fast, and Safe Access Control. AWS Open Source Blog, describing the design goals the engine used here inherits.
<https://aws.amazon.com/blogs/opensource/>
3. Amazon Verified Permissions. A production deployment of Cedar, evidence that the policy model scales beyond a local harness.
<https://aws.amazon.com/verified-permissions/>
4. Anthropic. Claude Code hooks reference. The lifecycle surface this harness attaches to (PreToolUse, SessionStart, PreCompact, Stop).
<https://docs.claude.com/en/docs/claude-code/hooks>
5. OWASP Top 10 for Large Language Model Applications. Prompt injection and excessive agency are the two categories this work

targets directly.

<https://owasp.org/www-project-top-10-for-large-language-model-applications/>

6. NIST AI Risk Management Framework (AI RMF 1.0). The govern/map/measure/manage decomposition this project's structure echoes.

<https://www.nist.gov/itl/ai-risk-management-framework>

7. Architecture Decision Records, M. Nygard. The ADR format used in methodology/decisions/.

<https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>